

AI Application Architecture

AI Workspace — Request/Response Flow | Assignment 5

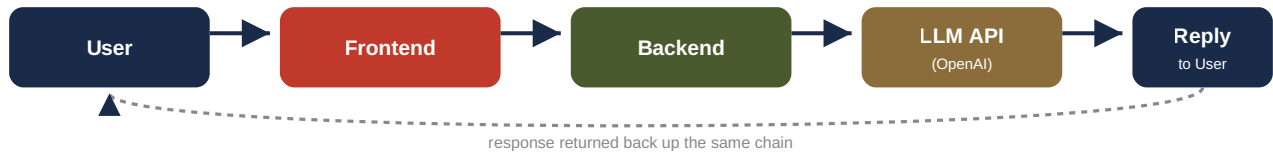


Figure 1 — High-level request/response flow of the AI Workspace application.

1. Components

- **User** — types a message into the chat box in the browser.
- **Frontend** — the Streamlit UI (`app.py`). Renders the chat, sidebar controls (system prompt, model, templates), and displays results.
- **Backend** — logic embedded in the same Streamlit process: builds the message list (system prompt + history + new message), applies the chosen prompt template, and manages session state.
- **LLM API** — OpenAI's `chat.completions` endpoint, which runs the actual language model and generates a reply.

2. Request Flow

1. User types a message and presses Enter → 2. Frontend captures the input via `st.chat_input()` → 3. Backend wraps it in the selected prompt template (if any) and prepends the system prompt + conversation history → 4. Backend calls `client.chat.completions.create()`, sending this full message list to the LLM API over HTTPS.

3. Response Flow

1. The LLM API returns a JSON object containing the generated text and token-usage stats → 2. Backend extracts `response.choices[0].message.content` → 3. Frontend renders it with `st.markdown()` so formatting (bold, lists, code blocks) displays correctly → 4. The exchange is appended to `st.session_state`, which is what gives the app conversation memory for the rest of the session.

4. Error Handling

Empty prompt: checked before any API call — shows a warning and stops.

Invalid/missing API key: caught via a `try/except` block around the API call; a 401-style error is translated into a friendly "check your API key" message.

Connection failure: the same `except` block catches timeouts/network errors and tells the user to check their internet connection, instead of the app crashing.

5. API Integration

The app uses the official `openai` Python SDK. The API key is read from an environment variable (`.env` file, via `python-dotenv`) or entered directly in the sidebar — it is never hard-coded into the source, which is a basic but important security practice. Each request sends the **model name**, the **message list**, and a **temperature** value (0.7, balancing creativity vs. consistency) to the API, and receives structured JSON back.